



Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра информационной безопасности

Суперкомпьютерное решение задачи разностной схемы для уравнения Пуассона задачи Дирихле

ОТЧЕТ ПО ВЫПОЛНЕНИЮ ЗАДАНИЙ ПО КУРСУ
«СУПЕРКОМПЬЮТЕРНОЕ МОДЕЛИРОВАНИЕ И ТЕХНОЛОГИИ»

Студент дневного отделения магистратуры

Лю Хайлинь

Преподаватель:

Доцент, Попова Н.Н.

Москва, 2025

Содержание

1	Введение	3
1.1	Актуальность	3
1.2	Постановка задачи	3
2	Математическая постановка и численные методы	5
2.1	Основное уравнение	5
2.2	Метод фиктивных областей	5
2.3	Метод конечных разностей	6
2.4	Метод сопряжённых градиентов с предобуславливанием	7
3	Программная реализация и результатов выполнения	9
3.1	Последовательная	9
3.2	Распараллеливание с OpenMP	9
3.3	Распараллеливание с MPI	13
3.4	Распараллеливание с MPI + OpenMP	15
3.5	Распараллеливание с MPI + CUDA	17
3.6	Анализ производительности	21
4	Заключение	26

1 Введение

1.1 Актуальность

Решение уравнения Пуассона имеет фундаментальное значение в математическом моделировании процессов теплопередачи, электростатического потенциала, диффузии и гидродинамики. Для большинства реальных задач, где область имеет сложную геометрию или неоднородные граничные условия, аналитические решения отсутствуют, поэтому требуется численное моделирование.

Современные вычислительные методы и технологии параллельных вычислений позволяют эффективно решать краевые задачи больших размерностей. Применение параллельных моделей, таких как OpenMP и MPI, обеспечивает значительное ускорение вычислений на многоядерных и многопроцессорных архитектурах.

Дополнительный рост производительности достигается за счёт использования графических ускорителей (GPU). Архитектура CUDA предоставляет программную модель с тысячами параллельных потоков и значительно более высокой пропускной способностью памяти по сравнению с традиционными CPU. Благодаря этому, GPU особенно эффективны при расчётах, содержащих регулярные сеточные структуры и большие объёмы однотипных операций — таких как вычисление дискретного лапласиана и обновление векторов в методе сопряжённых градиентов.

Таким образом, исследование производительности различных параллельных подходов — OpenMP, MPI и MPI+CUDA — представляет собой актуальную задачу суперкомпьютерных технологий. Сравнительный анализ позволяет оценить влияние архитектуры вычислительной системы на эффективность решения уравнения Пуассона и определить оптимальную стратегию распараллеливания в зависимости от доступных вычислительных ресурсов.

1.2 Постановка задачи

Цели работы

- Разработать и реализовать численный метод решения уравнения Пуассона в трапециевидной области с помощью метода конечных разностей.

- Исследовать эффективность параллельных реализаций на основе OpenMP, MPI и гибридной модели MPI+OpenMP.
- Реализовать версию программы на GPU с использованием CUDA и провести анализ её производительности.
- Провести сравнительный анализ ускорений и масштабируемости разработанных реализаций.

Задачи работы

- Построить разностную схему для уравнения Пуассона с использованием метода фиктивных областей.
- Реализовать итерационный метод сопряжённых градиентов для решения полученной системы линейных уравнений.
- Реализовать параллельные версии программы с применением OpenMP, MPI, гибридной схемы MPI+OpenMP и модели MPI+CUDA.
- Провести вычислительные эксперименты для различных размеров сеток и числа процессов/устройств.
- Проанализировать влияние параллелизма и архитектуры вычислителей на скорость сходимости, производительность и эффективность различных реализаций.

2 Математическая постановка и численные методы

2.1 Основное уравнение

Рассматривается двумерное уравнение Пуассона задачи Дирихле:

$$-\Delta u = f(x, y), \quad (x, y) \in D, \quad (2.1)$$

где Δ — оператор Лапласа:

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}. \quad (2.2)$$

Для заданной области D с границей ∂D формулируются граничные условия Дирихле:

$$u(x, y) = 0, \quad (x, y) \in \partial D. \quad (2.3)$$

Правая часть уравнения полагается постоянной:

$$f(x, y) = 1. \quad (2.4)$$

Область D представляет собой трапецию с вершинами

$$A(0, 0), \quad B(3, 0), \quad C(2, 3), \quad D(0, 3),$$

вложенную в прямоугольник $\Pi = [0, 3] \times [0, 3]$.

2.2 Метод фиктивных областей

Для нахождения приближённого решения задачи (2.1)–(2.3) используется метод конечных разностей. Поскольку рассматриваемая область D имеет сложную форму (трапецию), прямое построение сетки, совпадающей с её границей, является затруднительным. Для этого применяется **метод фиктивных областей**, позволяющий рассматривать область D как часть прямоугольной области Π , в которой удобно строить регулярную сетку.

Пусть область D принадлежит прямоугольнику Π , заданному как

$$\Pi = \{(x, y) : A_1 < x < B_1, A_2 < y < B_2\},$$

а $\hat{D} = \Pi \setminus D$ — фиктивная область (область вне D , но внутри Π).

Тогда исходное уравнение (2.1) переписывается в обобщённом виде:

$$-\frac{\partial}{\partial x} \left(k(x, y) \frac{\partial v}{\partial x} \right) - \frac{\partial}{\partial y} \left(k(x, y) \frac{\partial v}{\partial y} \right) = F(x, y), \quad (x, y) \in \Pi, \quad (2.5)$$

с граничным условием:

$$v(x, y) = 0, \quad (x, y) \in \Gamma = \partial\Pi, \quad (2.6)$$

где $k(x, y)$ — коэффициент теплопроводности, а $F(x, y)$ — обобщённая правая часть, определяемые следующим образом:

$$k(x, y) = \begin{cases} 1, & (x, y) \in D, \\ \frac{1}{\varepsilon}, & (x, y) \in \widehat{D}, \end{cases} \quad F(x, y) = \begin{cases} f(x, y), & (x, y) \in D, \\ 0, & (x, y) \in \widehat{D}, \end{cases} \quad (2.7)$$

где ε — малый параметр, обеспечивающий плавное выключение уравнения в фиктивной области.

2.3 Метод конечных разностей

Для построения численного решения уравнения (2.5) используется **метод конечных разностей**, который заключается в аппроксимации производных разностями на равномерной прямоугольной сетке.

В прямоугольной области $\Pi = [A_1, B_1] \times [A_2, B_2]$ задаётся равномерная сетка

$$x_i = A_1 + ih_x, \quad i = 0, 1, \dots, M, \quad y_j = A_2 + jh_y, \quad j = 0, 1, \dots, N, \quad (2.8)$$

где шаги сетки определяются как

$$h_x = \frac{B_1 - A_1}{M}, \quad h_y = \frac{B_2 - A_2}{N}.$$

В каждой внутренней узловой точке (x_i, y_j) производные в уравнении (2.5) аппроксимируются центральными разностями. Получаем разностную схему в виде:

$$\begin{aligned} & -\frac{1}{h_x^2} \left(k_{i+\frac{1}{2},j} (u_{i+1,j} - u_{i,j}) - k_{i-\frac{1}{2},j} (u_{i,j} - u_{i-1,j}) \right) \\ & -\frac{1}{h_y^2} \left(k_{i,j+\frac{1}{2}} (u_{i,j+1} - u_{i,j}) - k_{i,j-\frac{1}{2}} (u_{i,j} - u_{i,j-1}) \right) = F_{i,j}. \end{aligned} \quad (2.9)$$

где $u_{i,j} \approx u(x_i, y_j)$, а коэффициенты $k_{i\pm\frac{1}{2},j}$ и $k_{i,j\pm\frac{1}{2}}$ вычисляются как средние значения $k(x, y)$ между соседними узлами:

$$k_{i+\frac{1}{2},j} = \frac{1}{2} (k_{i,j} + k_{i+1,j}), \quad k_{i,j+\frac{1}{2}} = \frac{1}{2} (k_{i,j} + k_{i,j+1}). \quad (2.10)$$

Правая часть $F_{i,j}$ определяется как значение функции $F(x, y)$ в узле (x_i, y_j) :

$$F_{i,j} = F(x_i, y_j). \quad (2.11)$$

На границе $\Gamma = \partial\Pi$ реализуются граничные условия Дирихле:

$$u_{i,j} = 0, \quad (x_i, y_j) \in \Gamma. \quad (2.12)$$

Таким образом, для всех внутренних точек сетки $(i = 1, \dots, M - 1, j = 1, \dots, N - 1)$ уравнение (2.9) задаёт систему линейных алгебраических уравнений относительно неизвестных $u_{i,j}$:

$$A u = F,$$

где A — дискретный оператор Лапласа, модифицированный коэффициентами $k(x, y)$, а F — вектор правых частей.

2.4 Метод сопряжённых градиентов с предобуславливанием

После дискретизации получаем СЛАУ

$$A u = F, \quad A = A^\top \succ 0, \quad (2.13)$$

эквивалентную задаче минимизации квадратичного функционала

$$J(u) = \frac{1}{2} (A u, u) - (F, u). \quad (2.14)$$

Градиент функционала равен

$$\nabla J(u) = A u - F = -(F - A u) = -r, \quad r := F - A u, \quad (2.15)$$

поэтому движение в направлении r (или его предобусловленного варианта $z = D^{-1}r$) уменьшает J .

Для ускорения сходимости применяется диагональный предобуславливатель $D \approx A$:

$$(D w)_{i,j} = \left(\frac{k_{i+1,j} + k_{i,j}}{h_x^2} + \frac{k_{i,j+1} + k_{i,j}}{h_y^2} \right) w_{i,j}, \quad 1 \leq i \leq M - 1, \quad 1 \leq j \leq N - 1, \quad (2.16)$$

что даёт простое обращение $z = D^{-1}r$ покомпонентно.

Строить направления $p^{(k)}$, попарно A -сопряжённые,

$$(A p^{(k)}, p^{(\ell)}) = 0, \quad k \neq \ell, \quad (2.17)$$

и искать $u^{(k+1)} = u^{(k)} + \alpha_k p^{(k)}$ с оптимальным шагом

$$\alpha_k = \frac{(r^{(k)}, z^{(k)})}{(Ap^{(k)}, p^{(k)})}, \quad Dz^{(k)} = r^{(k)}. \quad (2.18)$$

Обновление направления выполняется по формуле

$$\beta_k = \frac{(r^{(k)}, z^{(k)})}{(r^{(k-1)}, z^{(k-1)})}, \quad p^{(k)} = z^{(k)} + \beta_k p^{(k-1)}. \quad (2.19)$$

После шага α_k остаток обновляется как

$$r^{(k+1)} = r^{(k)} - \alpha_k Ap^{(k)}. \quad (2.20)$$

Критерий остановки берём, например, по норме остатка:

$$\|r^{(k)}\|_2 \leq \varepsilon \quad \text{или} \quad \frac{\|r^{(k)}\|_2}{\|F\|_2} \leq \varepsilon_{\text{rel}}. \quad (2.21)$$

Задача решаем по следующему алгоритму:

Algorithm 1 Предобусловленный метод сопряжённых градиентов (PCG)

Require: $A, F, u^{(0)}, D, \varepsilon, k_{\max}$

```

1:  $r^{(0)} \leftarrow F - Au^{(0)}$ 
2:  $z^{(0)} \leftarrow D^{-1}r^{(0)}$ 
3:  $p^{(0)} \leftarrow z^{(0)}$ 
4:  $\rho^{(0)} \leftarrow (r^{(0)}, z^{(0)})$ 
5: for  $k = 0, 1, 2, \dots, k_{\max} - 1$  do
6:    $q \leftarrow Ap^{(k)}$ 
7:    $\alpha_k \leftarrow \rho^{(k)} / (p^{(k)}, q)$ 
8:    $u^{(k+1)} \leftarrow u^{(k)} + \alpha_k p^{(k)}$ 
9:    $r^{(k+1)} \leftarrow r^{(k)} - \alpha_k q$ 
10:  if  $\|r^{(k+1)}\|_2 \leq \varepsilon$  then
11:    break
12:  end if
13:   $z^{(k+1)} \leftarrow D^{-1}r^{(k+1)}$ 
14:   $\rho^{(k+1)} \leftarrow (r^{(k+1)}, z^{(k+1)})$ 
15:   $\beta_{k+1} \leftarrow \rho^{(k+1)} / \rho^{(k)}$ 
16:   $p^{(k+1)} \leftarrow z^{(k+1)} + \beta_{k+1}p^{(k)}$ 
17: end for
18: return  $u^{(k+1)}$ 

```

3 Программная реализация и результатов выполнения

3.1 Последовательная

На основе описанных выше методов реализован решатель для уравнения Пуассона с краевыми условиями Дирихле. Решатель построен на использовании класса `PoissonSolver`, определённого в файле `include/conjugate_gradient.hpp`. Данный класс реализует все этапы метода предобусловленного сопряжённого градиента (PCG), включая построение сетки, инициализацию, применение оператора A , предобуславливание и итерационный процесс поиска решения. Метод конечных разностей применяется для аппроксимации оператора Лапласа, а фиктивные области позволяют обрабатывать граничные участки трапециевидной геометрии. Итерации выполняются до достижения нормы остатка, меньшей заданного допуска ε .

Основной файл `task.cpp` организует ввод параметров сетки (M, N) , запуск решателя и измерение времени работы программы. После завершения вычислений решение сохраняется в виде CSV-файла.

При $(M, N) = (10, 10), (20, 20), (40, 40)$ используем последовательную программу для вычисления приближенного решения. Получены следующие результаты:

Сетка ($M \times N$)	Число итераций	Время(s)
(10×10)	20	0.000837
(20×20)	47	0.007440
(40×40)	108	0.069108

Таблица 1: Последовательная программа на разных сеток

3.2 Распараллеливание с OpenMP

В дальнейшем на основе последовательной реализации была добавлена поддержка параллельных вычислений с использованием технологии `OpenMP`. Основная идея заключалась в распараллеливании наиболее трудоёмких двойных циклов по индексам (i, j) , возникающих при вычислении операторов Au , Ap , а также при обновлениях векторов u , r , p и z . Для этого в код были добавлены директивы `#pragma omp parallel for collapse(2)`,

позволяющие автоматически разделять итерации по потокам. Такое решение обеспечивает равномерную загрузку вычислительных ядер и значительно ускоряет процесс решения при больших размерах сетки.

Кроме того, в местах, где использовались операции суммирования (например, при вычислении норм и скалярных произведений), были добавлены конструкции `reduction(+:sum)`, что позволило корректно и безопасно объединять результаты вычислений от разных потоков. Таким образом, все основные участки с независимыми вычислениями были распараллелены, при этом численная точность метода и устойчивость итерационного процесса сохраняются.

При $(M, N) = (40, 40)$ используем последовательную и параллельную программу с 1, 4, 16-нитей. Получены следующие результаты.

Количество нитей	Сетка ($M \times N$)	Число итераций	Время (s)	Ускорение
-	(40×40)	108	0.069108	1.00
1	(40×40)	108	0.069582	1.00
4	(40×40)	108	0.020581	3.36
16	(40×40)	108	0.012877	5.37

Таблица 2: Распараллеливание с OpenMP с разными количествами нитей

Как показано в таблице 2, при небольших размерах сетки (M, N) объём вычислений остаётся относительно малым, поэтому использование параллельных программ OpenMP не приводит к заметному ускорению по сравнению с последовательной версией. Это связано с тем, что при малом количестве узлов затраты на создание потоков, их синхронизацию и переключение между ними становятся соизмеримыми с самим временем вычислений, что снижает общую эффективность параллелизации.

При увеличении размеров задачи влияние накладных расходов уменьшается, и преимущества параллельных вычислений становятся более очевидными. В дальнейшем мы рассмотрим результаты для более крупных сеток $(M, N) = (400, 600)$ и $(800, 1200)$, где применение OpenMP демонстрирует существенное ускорение по сравнению с последовательной реализацией.

Количество нитей	Сетка ($M \times N$)	Число итераций	Время (s)	Ускорение
-	(400×600)	1834	10.3237	1.00
2	(400×600)	1834	5.34118	1.93
4	(400×600)	1834	2.77427	3.72
8	(400×600)	1834	1.55052	6.66
16	(400×600)	1812	1.16976	8.83
-	(800×1200)	3759	85.0672	1.00
4	(800×1200)	3759	21.6282	3.93
8	(800×1200)	3759	11.2939	7.53
16	(800×1200)	3759	7.08205	12.01
32	(800×1200)	3759	5.59543	15.20

Таблица 3: Распараллеливание с OpenMP при больших размеров сетки

Анализ представленных данных показывает, что при увеличении размеров сетки эффективность распараллеливания возрастает значительно. Для сетки (400×600) ускорение при 16 потоках достигает значения $S = 8.83$, что близко к линейному масштабированию. При дальнейшем увеличении задачи до (800×1200) достигается ускорение $S = 15.20$ при 32 потоках, что указывает на хорошую масштабируемость метода и эффективное использование вычислительных ресурсов.

Такое поведение объясняется тем, что при больших размерах задачи доля вычислений в общем времени исполнения резко возрастает, а накладные расходы, связанные с управлением потоками, становятся незначительными. Таким образом, реализованный OpenMP-решатель демонстрирует высокую эффективность на крупномасштабных сетках и способен обеспечивать почти идеальную параллельную производительность при увеличении числа вычислительных ядер.

Для наглядного представления полученного приближённого решения уравнения Пуассона наибольшего размера сетки $(M, N) = (800, 1200)$ были построены визуализации в двумерном и трёхмерном форматах. На рисунке 1 показано распределение значений функции $u(x, y)$ на плоскости, что позволяет оценить структуру решения и поведение функции внутри области. Трёхмерная визуализация, представленная на рисунке 2, иллюстрирует рельеф поверхности решения и демонстрирует плавное изменение потенциала по области.

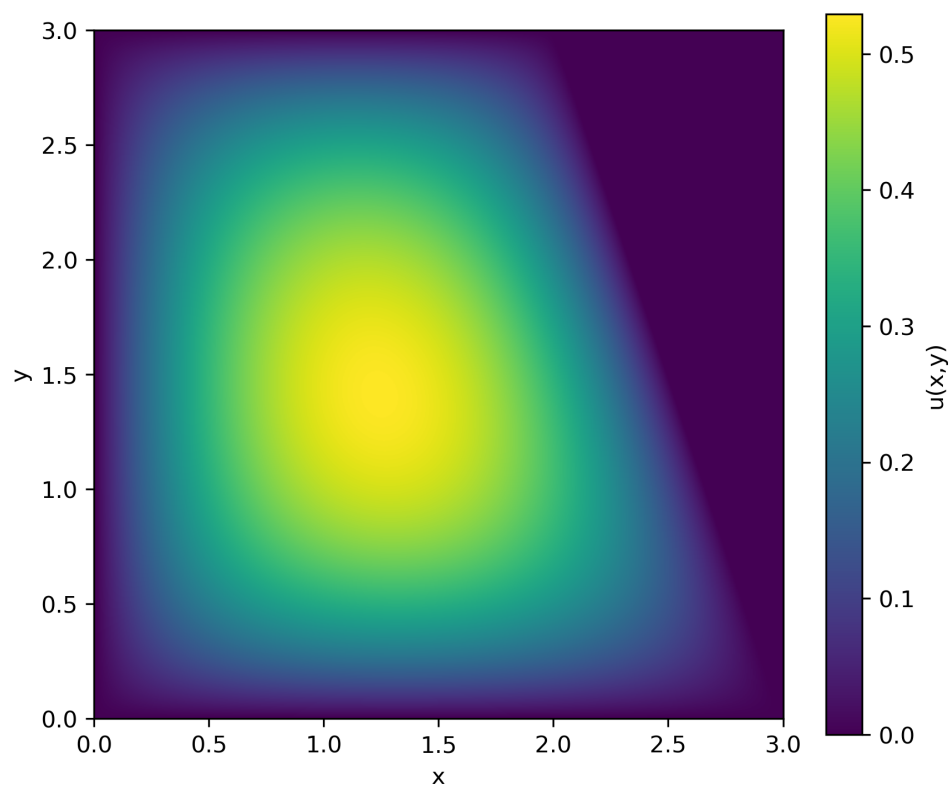


Рис. 1: Двумерная визуализация решения для сетки (800×1200)

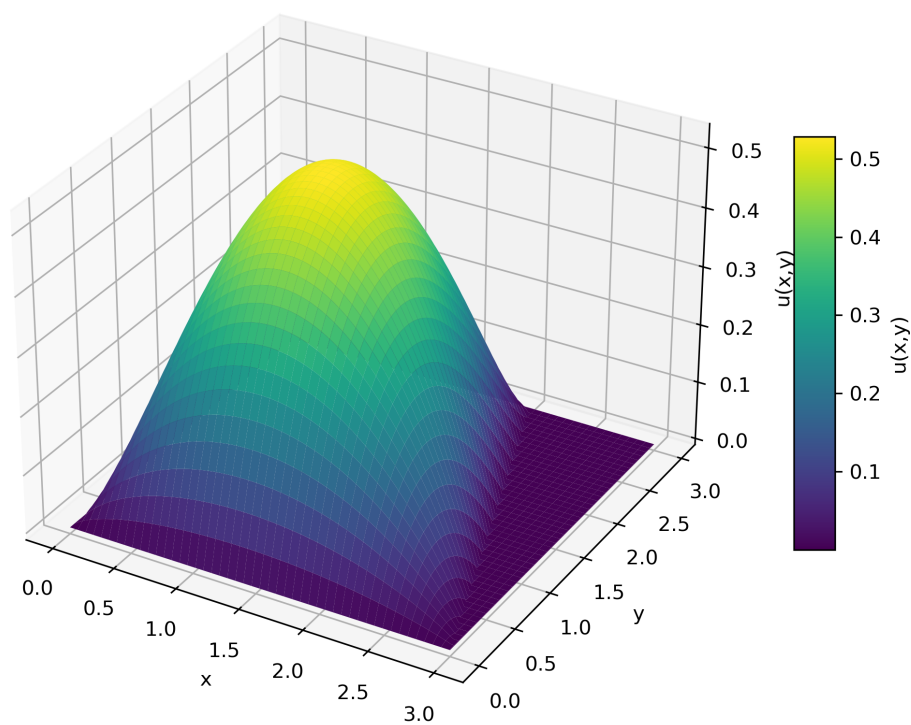


Рис. 2: Трёхмерная визуализация решения для сетки (800×1200)

3.3 Распараллеливание с MPI

Для распределения глобальной прямоугольной сетки по процессам реализован класс `DomainDecomposer`, который строит блочную декомпозицию $p_x \times p_y$ и удовлетворяет двум условиям: (1) отношение сторон каждого локального блока не превышает 2:1, (2) длины подотрезков по каждой оси отличаются не более чем на 1 узел. Для этого выбираются p_x, p_y как близкие к $\sqrt{P}$ множители степени двойки с учётом соотношения $M:N$, а разбиение по каждой оси выполняется как равномерное (блоки размера $\lfloor \frac{\text{size}}{\text{blocks}} \rfloor$ или на единицу больше).

На построенной декомпозиции создаётся декартова топология процессов `MPI_Cart_create` с размерами (p_x, p_y) и без периодичности. Каждому рангу сопоставляются координаты `coords` и его локальный прямоугольник индексов $[x_{\min} : x_{\max}] \times [y_{\min} : y_{\max}]$ (в глобальной нумерации). Далее для каждого процесса конструируется свой `MPIPoissonSolver` на локальной области (с добавлением гало-слоёв по соседям), после чего выполняется итерационный метод.

Ниже показан пример декомпозиции для сетки 800×1200 при $P = 16$:

Пример декомпозиции области для 800×1200 , $P = 16$

```
==== Domain Decomposition Summary ====
Global grid: 800 x 1200
Total processes: 16    Grid layout: 4 x 4
X partitions (4):      0 200 400 600
Y partitions (4):      0 300 600 900
=====
[Rank 0] coords=(0,0) x:[  0, 199] y:[  0, 299]
[Rank 1] coords=(0,1) x:[  0, 199] y:[ 300, 599]
[Rank 2] coords=(0,2) x:[  0, 199] y:[ 600, 899]
[Rank 3] coords=(0,3) x:[  0, 199] y:[ 900,1200]
[Rank 4] coords=(1,0) x:[ 200, 399] y:[  0, 299]
[Rank 5] coords=(1,1) x:[ 200, 399] y:[ 300, 599]
[Rank 6] coords=(1,2) x:[ 200, 399] y:[ 600, 899]
[Rank 7] coords=(1,3) x:[ 200, 399] y:[ 900,1200]
[Rank 8] coords=(2,0) x:[ 400, 599] y:[  0, 299]
[Rank 9] coords=(2,1) x:[ 400, 599] y:[ 300, 599]
[Rank 10] coords=(2,2) x:[ 400, 599] y:[ 600, 899]
[Rank 11] coords=(2,3) x:[ 400, 599] y:[ 900,1200]
[Rank 12] coords=(3,0) x:[ 600, 800] y:[  0, 299]
[Rank 13] coords=(3,1) x:[ 600, 800] y:[ 300, 599]
[Rank 14] coords=(3,2) x:[ 600, 800] y:[ 600, 899]
[Rank 15] coords=(3,3) x:[ 600, 800] y:[ 900,1200]
```

Класс `MPIPoissonSolver` наследует базовый класс `PoissonSolver` и добавляет распределённые операции на основе `MPI`. В конструкторе создаётся декартова топология процессов, а для каждого процесса определяются соседи по четырём направлениям: `left`, `right`, `bottom`, `top`. Это позволяет эффективно организовать обмен граничными слоями (halo exchange) между соседними поддоменами.

Обмен граничными слоями. Для корректного вычисления разностного оператора Лапласа вблизи границ локальной области каждая подзадача хранит дополнительные строки и столбцы «призрачные» узлы. Метод `exchange_halo()` выполняет асинхронный обмен этими данными между соседними процессами с помощью пар вызовов `MPI_Isend/MPI_Irecv` и последующего ожидания `MPI_Waitall`. После завершения обмена граничные значения копируются в соответствующие ячейки сетки, обеспечивая непрерывность решения на стыках поддоменов.

Редукция и вычисление глобальных норм. Поскольку каждая подзадача хранит только часть сетки, скалярные произведения и нормы (например, $\|r\|_2$ или (r, z)) вычисляются локально и затем объединяются во всей топологии с помощью `MPI_Allreduce`.

Завершение и сборка решения. После завершения итерационного процесса каждый процесс хранит свою часть приближённого решения $u(x, y)$. Далее с помощью операции `MPI_Gatherv` данные передаются на `rank 0`, который собирает их в единую глобальную матрицу решения.

Мы также протестировали производительность `MPI` на малых и больших сетках:

Количество процессов	Сетка ($M \times N$)	Число итераций	Время (s)	Ускорение
(Без <code>MPI</code>)	(40×40)	108	0.069108	1.00
1	(40×40)	108	0.069582	1.00
2	(40×40)	108	0.036846	1.88
4	(40×40)	108	0.020216	3.42

Таблица 4: Распараллеливание с `MPI` с разными количествами процессов

Количество процессов	Сетка ($M \times N$)	Число итераций	Время (s)	Ускорение
1	(400×600)	1834	8.56922	1.00
2	(400×600)	1834	4.55551	1.88
4	(400×600)	1834	2.44147	3.51
8	(400×600)	1834	1.47967	5.79
16	(400×600)	1816	1.24117	6.90
1	(800×1200)	3759	71.3076	1.00
4	(800×1200)	3737	18.0492	3.95
8	(800×1200)	3737	9.61040	7.42
16	(800×1200)	3759	5.71856	12.47
32	(800×1200)	3759	4.91637	14.50

Таблица 5: Распараллеливание с MPI при больших размеров сетки

Анализ экспериментальных результатов показывает, что MPI-параллелизация демонстрирует высокую эффективность и почти линейное ускорение при увеличении числа процессов. Для небольшой сетки (40×40) ускорение ограничено, поскольку объём вычислений невелик и накладные расходы на коммуникацию между процессами сопоставимы с самим временем вычислений. Однако уже при размере (400×600) наблюдается почти идеальная масштабируемость: при 16 процессах ускорение достигает $S = 6.90$.

Для ещё более крупной сетки (800×1200) поведение остаётся аналогичным: при 32 процессах достигается ускорение $S = 14.50$, что подтверждает отличную масштабируемость решателя и эффективность схемы обмена граничными слоями. Незначительные расхождения в числе итераций (в пределах нескольких шагов) связаны с особенностями вычислений в плавающей арифметике: порядок операций при редукции скалярных произведений в MPI может незначительно влиять на накопленную погрешность и, соответственно, на критерий сходимости.

3.4 Распараллеливание с MPI + OpenMP

Для дальнейшего повышения производительности комбинируется распределённая (MPI) и потоковая (OpenMP) параллелизация. В этой гибридной схеме каждая подзадача,

запущенная на своём MPI-процессе, дополнительно использует несколько потоков OpenMP для распараллеливания вложенных циклов по узлам сетки. Такой подход позволяет эффективно задействовать как межузловой, так и внутрипроцессорный параллелизм.

Процессы	Нити	Сетка	Число итераций	Время (s)	Ускорение
-	-	(40×40)	108	0.069108	1.00
1	4	(40×40)	108	0.036172	1.91
2	4	(40×40)	108	0.020501	3.37

Таблица 6: Распараллеливание с MPI+OpenMP с разными количествами процессов и нитей

Процессы	Нити	Сетка	Число итераций	Время (s)	Ускорение
-	-	(400×600)	1834	8.56922	1.00
2	1	(400×600)	1834	5.45576	1.57
2	2	(400×600)	1834	3.77839	2.27
2	4	(400×600)	1834	3.34283	2.56
2	8	(400×600)	1834	3.39928	2.52
-	-	(800×1200)	3759	71.3076	1.00
4	1	(800×1200)	3737	21.4454	3.33
4	2	(800×1200)	3737	14.6345	4.87
4	4	(800×1200)	3756	12.8286	5.56
4	8	(800×1200)	3756	13.0299	5.47

Таблица 7: Распараллеливание с MPI+OpenMP при больших размерах сетки

Как видно из представленных данных, добавление OpenMP-параллелизма поверх MPI обеспечивает дополнительное ускорение, особенно при увеличении числа нитей. Для небольшой сетки (40×40) выигрыш умеренный, однако при увеличении размеров задачи (400×600) и (800×1200) достигается устойчивое ускорение около 2.5 - 5.5 раз при использовании 8 потоков.

3.5 Распараллеливание с MPI + CUDA

Для дальнейшего повышения производительности была реализована версия решателя на основе гибридной модели **MPI + CUDA**. В этой схеме декомпозиция области и коммуникации между процессами выполняются средствами MPI (класс `MPIPoissonSolver`), а наиболее трудоёмкие вычислительные операции переносятся на графические ускорители через класс `MPI CUDA Poisson Solver`. Реализация организована в виде трёхслойной структуры:

В основе CUDA-реализации лежит трёхуровневая архитектура, включающая набор низкоуровневых CUDA-ядер, систему CUDA-операторов и высокоуровневый класс `MPI CUDA Poisson Solver`. На первом уровне расположены CUDA-ядра (файлы `cuda_kernels.cu/cuda_kernels.cuh`), выполняющие все вычисления на сетке. К ним относятся ядро `apply_A_kernel`, вычисляющее дискретный дифференциальный оператор $Au = -\nabla \cdot (k \nabla u)$, а также векторные ядра `update_u_kernel`, `update_r_kernel`, `update_p_kernel`, `apply_preconditioner_kernel`, реализующие операции обновления, характерные для метода сопряжённых градиентов. Дополнительно используются ядра `compute_r2_kernel`, `compute_p_Ap_kernel`, `compute_rz_kernel`, которые формируют поэлементные квадраты или произведения векторов для последующей редукции, и ядро `reduce_pairwise_kernel`, выполняющее попарную параллельную редукцию одномерного массива. Все ядра работают на прямоугольной сетке потоков и блоков, используют индексацию `idx(i,j,N+1)` и исключают граничные узлы, зануляя соответствующие элементы в буфере.

Второй уровень реализован в виде набора CUDA-операторов (файл `cuda_operators.cu`), представляющих собой удобные оболочки над ядрами. Они формируют параметры запуска `dim3 grid, block` исходя из локального размера поддомена, принимают CUDA-поток `cudaStream_t stream` для управления асинхронным выполнением, автоматически выполняют проверку ошибок и предоставляют унифицированный интерфейс к операциям. Через эти функции вызываются все ядра: `cuda_apply_A`, `cuda_update_u`, `cuda_update_r`, `cuda_update_p`, `cuda_apply_preconditioner`, а также вычислительные операторы `cuda_compute_r2`, `cuda_compute_p_Ap`, `cuda_compute_rz`. Важной частью является оператор `cuda_pairwise_reduce`, который многократно запускает `reduce_pairwise_kernel`,

уменьшая длину массива вдвое на каждом шаге до требуемого размера, что позволяет минимизировать объём данных, пересылаемых с GPU на CPU.

Третий уровень `MPICudaPoissonSolver` (файл `mpi_cuda_conjugate_gradient.cpp`), наследующий `MPIPoissonSolver` и интегрирующий вычисления на GPU в MPI-ориентированный итерационный метод. В конструкторе выделяется память на устройстве для всех необходимых массивов (`d_u`, `d_r`, `d_z`, `d_p`, `d_A_p`, `d_k`, `d_M_inv`, `d_buf`), а также создаётся выделенный поток `cudaStream_t`. Методы `to_device` и `to_host` обеспечивают перенос данных между CPU и GPU через промежуточный буфер `h_buf`. Все ключевые функции базового решателя (`compute_l2_norm`, `compute_rz`, `compute_p_Ap`, `update_u`, `update_r`, `update_p`, `apply_preconditioner`, `solve`) переопределяются и реализуются при помощи CUDA-операторов. Отдельно реализован метод `cuda_exchange_halo()`, который копирует граничные слои с GPU на CPU, выполняет обмен через `MPI_Isend`/`MPI_Irecv` и затем возвращает обновлённые данные обратно на GPU, сохраняя совместимость с существующей MPI-топологией и полностью поддерживая вычисление оператора A на устройстве.

Типы операторов на GPU

В CUDA-версии метода используются три класса вычислительных операторов. Во-первых, полностью параллельные векторные операции, включающие обновление решения, остатка, направления поиска и применение предобуславливателя:

$$u^{(k+1)} = u^{(k)} + \alpha p^{(k)}, \quad r^{(k+1)} = r^{(k)} - \alpha A p^{(k)}, \quad p^{(k+1)} = z^{(k+1)} + \beta p^{(k)}, \quad z = M^{-1}r.$$

Каждая из этих формул реализуется отдельным CUDA-ядром `update_u_kernel`, `update_r_kernel`, `update_p_kernel`, `apply_preconditioner_kernel`. Все операции выполняются независимо для каждого внутреннего узла сетки, что обеспечивает их естественный и полный параллелизм.

Второй тип вычислений дискретный оператор Лапласа, где коэффициенты на полуцелых узлах вычисляются как среднее ближайших значений k . Эта операция реализована в ядре `apply_A_kernel`, вызываемом через интерфейс `cuda_apply_A`. Поскольку оператор использует только локальные соседние значения, вычисления полностью распараллеливаются по узлам сетки и хорошо масштабируются на GPU.

Третий класс операторы редукции, используемые для вычисления норм и скалярных произведений:

$$\|r\|_2^2 = \sum_{i,j} r_{ij}^2, \quad (p, Ap) = \sum_{i,j} p_{ij}(Ap)_{ij}, \quad (r, z) = \sum_{i,j} r_{ij}z_{ij}.$$

Для этих операций используется мультиэтапная схема: сначала GPU формирует массив частичных произведений при помощи ядер `compute_r2_kernel`, `compute_p_Ap_kernel`, `compute_rz_kernel`, затем выполняется иерархическая попарная редукция через `reduce_pairwise_kernel`. После уменьшения массива до небольшого размера данные копируются на CPU, где выполняется финальная локальная редукция и глобальная MPI-редукция `MPI_Allreduce`. Такой подход минимизирует объём обмена между устройством и хостом и обеспечивает детерминированный результат во всех конфигурациях GPU.

Трёхэтапная схема редукции

Редукция норм и скалярных произведений в CUDA-версии реализована в три последовательных шага. На первом этапе на GPU формируется массив поэлементных вкладов: в зависимости от конкретной величины запускается одно из ядер `compute_r2_kernel`, `compute_p_Ap_kernel` или `compute_rz_kernel`, которые записывают в буфер `d_buf` соответственно значения r_{ij}^2 , $p_{ij}(Ap)_{ij}$ или $r_{ij}z_{ij}$ для всех внутренних узлов локальной области (граничные точки зануляются). Таким образом, `d_buf` содержит плотный одномерный массив длины $(M+1)(N+1)$ с уже подготовленными слагаемыми.

На втором этапе на устройстве применяется страйдовая редукция `cuda_pairwise_reduce`, которая многократно запускает ядро `reduce_pairwise_kernel`. На k -й итерации выбирается шаг s_k (обычно $s_k \approx \lfloor n_k/2 \rfloor$), и каждый поток с глобальным индексом `tid` при условии `tid + s_k < n_k` выполняет операцию `data[tid] += data[tid + s_k]`, то есть «добавляет» правую половину массива в соответствующие элементы левой половины. После такого прохода информативной остаётся только левая часть длиной $n_{k+1} \approx \lfloor n_k/2 \rfloor$, а элементы правой части более не используются. Цикл повторяется с новым шагом до тех пор, пока текущий размер массива не станет меньше заданного порога `target_n` (например, 1024). При этом каждое ядро записывает только в свой уникальный индекс, и в реализации принципиально НЕ используется разделяемая

память (`__shared__`) или warp-специфические примитивы: все операции выполняются исключительно в глобальной памяти с простыми чтениями и операциями `+=`.

На третьем этапе укороченный массив длиной $O(10^3)$ копируется с устройства на хост (`cudaMemcpy`), после чего оставшиеся элементы суммируются обычным циклом на CPU, формируя локальную скалярную величину. Далее для объединения вкладов от всех MPI-процессов выполняется глобальная редукция `MPI_Allreduce` по одному числу типа `double`. Таким образом, полные значения $\|r\|_2$, (p, Ap) и (r, z) вычисляются в три шага: GPU-предобработка и страйдовая редукция в глобальной памяти, компактная локальная редукция на CPU и финальная глобальная редукция в MPI, при этом условие задания о невозможности использования разделяемой памяти для реализации редукции строго соблюдается.

Данная схема позволяет существенно разгрузить CPU и сеть: большая часть работы по суммированию выполняется на GPU, а через шину CPU–GPU и по MPI-подсистеме передаётся только небольшое количество агрегированных данных.

Организация обмена граничными слоями на GPU

Для корректной работы оператора Лапласа на границах локальных областей требуется актуальная информация о соседних узлах. В CUDA-версии это реализовано через поэтапную схему обмена гало-слоями между устройством и MPI. Сначала данные граничных слоёв поля `d_p` (или другого массива) копируются с устройства на хост с помощью `cudaMemcpy` в специально выделенные буферы `send_left`, `send_right`, `send_top`, `send_bottom`. Затем на стороне CPU выполняется асинхронный обмен этими буферами с соседними рангами в декартовой топологии с использованием пар вызовов `MPI_Isend`/`MPI_Irecv`, после чего выполняется коллективное ожидание `MPI_Waitall`. Полученные от соседей данные помещаются в буферы `recv_left`, `recv_right`, `recv_top`, `recv_bottom` и далее копируются обратно на устройство в соответствующие гало-слои массива `d_p`. Такой подход позволяет без изменения логики MPI-версии поддерживать согласованность граничных значений между поддоменами, при этом основные вычисления оператора A и все векторные обновления выполняются целиком на GPU.

Сравнение производительности MPI и MPI+CUDA

Для оценки эффективности использования графических ускорителей была проведена серия экспериментов на сетках (400×600) и (800×1200) с одинаковой декомпозицией по MPI-процессам. В таблице ниже приведены результаты:

Процессы	GPU	Сетка	Время (s)	Итерации	Ускорение
1	-	(400×600)	10.3237	1834	1.00
1	1	(400×600)	1.22757	1834	8.41
2	2	(400×600)	1.22289	1834	8.44
4	4	(400×600)	1.24033	1834	8.32
1	-	(800×1200)	85.0672	3759	1.00
1	1	(800×1200)	5.63806	3759	15.09
2	2	(800×1200)	3.95125	3759	21.53
4	4	(800×1200)	3.38396	3737	24.14

Таблица 8: Результаты MPI+CUDA

Как видно, использование CUDA позволяет существенно сократить общее время решения за счёт переноса вычисления лапласиана, векторных обновлений и части редукций на GPU. Точная количественная оценка ускорения и детальный разбор временных затрат по категориям (инициализация, параллельные циклы, копирование данных, коммуникации, редукции) приводятся в следующих разделах отчёта.

3.6 Анализ производительности

В данном разделе рассматривается структура временных затрат для четырёх вариантов реализации решателя уравнения Пуассона (остальные в приложении отчёта): исходной последовательной программы на CPU, параллельной версии с использованием OpenMP, распределённой версии на основе MPI и гибридной реализации MPI+CUDA, в которой наиболее трудоёмкие части перенесены на графические ускорители. Сопоставление проводится как по общему времени работы, так и по вкладу трёх ключевых групп вычислительных операторов (дискретный оператор Лапласа, полностью параллельные

векторные обновления и редукции норм/скалярных произведений), а также по дополнительным затратам на инициализацию, завершение, обмен граничными слоями и, в GPU-версии, копирование данных между хостом и устройством. В качестве эталонного примера для детального анализа используется задача на сетке $(M, N) = (800 \times 1200)$ при степени параллелизма $P = 2$; для этого сценария ниже приведены помодульные профили временных затрат для всех четырёх реализаций, тогда как дополнительные графики масштабируемости и результаты для других конфигураций вынесены в приложения.

Для последовательной реализации на сетке (800×1200) более половины общего времени ($\approx 52\%$) приходится на применение оператора Лапласа, ещё около 27% занимают полностью параллельные векторные обновления и порядка 20% — редукции при вычислении норм и скалярных произведений. Инициализация и финализация суммарно дают менее 1% и практически не влияют на масштабируемость. Таким образом, для последовательной программы профиль временных затрат полностью определяется тремя основными типами операторов метода сопряжённых градиентов; именно они являются естественными кандидатами для последующей параллелизации.

Этап	Время, с	Доля, %
Инициализация	0.0565	0.07
Оператор Лапласа	44.2729	52.04
Операторы обновления	23.1509	27.20
Редукции	17.0781	20.08
Завершение	0.5276	0.62
Итого	85.0672	100.00

Таблица 9: Профиль временных затрат последовательной программы (800×1200)

При переходе к OpenMP-версии с двумя нитями общий runtime сокращается примерно вдвое: с 85.0672 с до 42.1849 с, причём ускорение для всех трёх групп операторов практически одинаково (≈ 2.0 раза). Время работы оператора Лапласа уменьшается до 22.1917 с, векторные обновления занимают порядка 11.1050 с, редукции — около 8.3164 с, при этом относительные доли этих трёх компонентов практически не меняются: оператор Лапласа остаётся основным «узким местом» (около 52% общего времени), а

вклад служебных этапов инициализации/финализации остаётся на уровне десятых долей процента. Это показывает, что потоковый параллелизм с OpenMP хорошо ускоряет наиболее тяжёлые вычислительные ядра, но не меняет качественную структуру профиля: производительность по-прежнему ограничивается стоимостью применения дискретного оператора Лапласа и пропускной способностью памяти.

Этап	Время, с	Доля, %
Инициализация	0.0323	0.08
Оператор Лапласа	22.1917	52.61
Операторы обновления	11.1050	26.33
Редукции	8.3164	19.72
Завершение	0.5294	1.26
Всего	42.1849	100.00

Таблица 10: Профиль временных затрат OpenMP-реализации (2 нити на 800×1200)

MPI-реализация при $P = 2$ демонстрирует очень похожую картину по основным вычислительным компонентам: около 40.00% времени тратится на оператор Лапласа, порядка 25.62% — на векторные обновления и около 32.00% — на локальные редукции до глобального объединения. При этом явно выделяются два дополнительных источника затрат, связанных с распределённой природой метода: обмен гало-слоями между соседними процессами (`MPI_Isend/MPI_Irecv + MPI_Waitall`) и глобальные редукции `MPI_Allreduce`. Их абсолютное время при $P = 2$ невелико (менее 0.03 с для halo-exchange и около 0.07 с для глобальных редукций, суммарно менее 0.3% общего времени), но именно эти операции становятся потенциальными «бутылочными горлышками» при масштабировании на большее число процессов, так как выполняются в каждой итерации метода сопряжённых градиентов.

Этап	Время, с	Доля, %
Инициализация	0.1371	0.38
Оператор Лапласа	14.3088	40.00
Операторы обновления	9.1638	25.62
Локальные редукции	11.4511	32.00
MPI halo exchange	0.0297	0.08
MPI Allreduce	0.0757	0.21
Завершение	0.5630	1.57
Всего	35.7825	100.00

Таблица 11: Профиль временных затрат MPI-реализации (800×1200) при $P = 2$

В гибридной реализации MPI+CUDA на той же сетке и при тех же двух MPI-процессах (таблица 12) ситуация принципиально меняется: общее время решения сокращается до 3.95 с, то есть на два порядка по сравнению с последовательной версией и на два порядка по сравнению с чисто MPI-вариантом. При этом доминирующим компонентом становятся локальные редукции, реализованные в три этапа (GPU-предредукция, страйдная попарная редукция на устройстве и финальное суммирование на CPU): совокупно они занимают около 34.1% общего времени, из которых примерно 32.95% приходится на CUDA-ядра редукции. Операторы Лапласа и обновления (в сумме 12.3%) становятся существенно дешевле, а коммуникационные этапы (MPI halo exchange, MPI Allreduce) и копирование данных между хостом и устройством (около 11.6%) остаются умеренными благодаря компактным буферам и трёхэтапной схеме редукции. Таким образом, в профиле MPI+CUDA основное «узкое место» переносится с оператора Лапласа и обменов по MPI на GPU-редукции, что соответствует ожидаемому сдвигу к вычислительно доминируемой конфигурации при использовании графических ускорителей.

Этап	Время, с	Доля, %
Инициализация	0.2718	6.88
Операторы Лапласа	0.3032	7.67
- CPU	0.0134	0.34
- CUDA	0.2898	7.33
Операторы обновления	0.1815	4.59
- CPU	0.0198	0.50
- CUDA	0.1617	4.09
Локальные редукции	1.3488	34.13
- CPU	0.0466	1.18
- CUDA	1.3022	32.95
Копирование Host→Device	0.0949	2.40
Копирование Device→Host	0.3624	9.17
MPI halo exchange	0.0215	0.54
MPI Allreduce	0.0675	1.71
Завершение	0.9407	23.81
Итого	3.95125	100.00

Таблица 12: Профиль временных затрат MPI+CUDA-реализации (800×1200 , 2 GPU)

4 Заключение

В данной работе был разработан и всесторонне исследован вычислительный комплекс для решения уравнения Пуассона в трапециевидной области методом предобусловленных сопряжённых градиентов (PCG). Главной особенностью проекта является систематическая эволюция от последовательной реализации к многоуровневым параллельным моделям: OpenMP, MPI и гибридной архитектуре MPI+CUDA. Такой подход позволил глубоко проанализировать влияние различных типов параллелизма на производительность и показать их сильные и слабые стороны в реальных вычислительных условиях.

Разработанная структура кода обладает высокой модульностью и расширяемостью. Благодаря классу `DomainDecomposer` реализовано автоматическое построение сбалансированных MPI-декомпозиций с соблюдением пропорций поддоменов, что обеспечивает стабильную масштабируемость на современных вычислительных кластерах. Поддержка гибридной модели MPI+CUDA реализована через класс `MPI CUDA Poisson Solver`, который дополняет базовую MPI-логику переносом ключевых вычислений на графический ускоритель. Трёхслойная архитектура CUDA — «ядра $\rightarrow$ операторы $\rightarrow$ высокоуровневый метод» — обеспечивает ясность, изоляцию уровней и удобство сопровождения.

Проведённый анализ производительности выявил важные особенности каждого из подходов. Последовательная программа ожидаемо демонстрирует доминирование оператора Лапласа и редукций в суммарном времени. Параллелизация с помощью OpenMP обеспечивает почти двукратное ускорение на двух потоках и существенно снижает стоимость всех основных операторов. MPI-реализация показывает высокую вычислительную эффективность и очень низкие коммуникационные издержки при малом числе процессов, однако наличие глобальных редукций ограничивает потенциальную масштабируемость при росте P .

Наиболее впечатляющие результаты достигаются в гибридной версии MPI+CUDA. На сетке (800×1200) время решения сокращается с порядка 35 секунд (MPI) и 71 секунд (последовательная версия) до 4 секунд при использовании двух GPU. При этом профиль временных затрат сдвигается в сторону вычислений на устройстве: оператор Лапласа и векторные обновления занимают лишь небольшую долю времени, а основным узким местом становятся редукции на GPU. Тем не менее даже с учётом этого фактора

MPI+CUDA реализует ускорение порядка 20+ раз относительно последовательной версии, что делает её наиболее продуктивным вариантом из всех реализованных.

Итогом работы стало создание законченного, надёжного и высокопроизводительного программного комплекса, включающего полный набор реализаций — CPU, OpenMP, MPI, MPI+OpenMP и MPI+CUDA — а также средства автоматизированного запуска и профилирования на суперкомпьютере. Полученные результаты демонстрируют, что сочетание строгой численной схемы, тщательно продуманной структуры кода и современных технологий параллельных вычислений позволяет достичь чрезвычайно высокой эффективности при моделировании уравнений математической физики на больших сетках. Такой комплекс может служить основой как для дальнейших научных исследований, так и для реальных инженерных приложений в области НРС.

Приложение А. Дополнительные результаты

В данном приложении приведены дополнительные материалы, демонстрирующие подробный профиль производительности решателя на двух типичных конфигурациях: сетке (400×600) при степенях параллелизма $P = 1, 2, 4$ и более крупной задаче (800×1200) при $P = 4$. Для каждой конфигурации представлены разложения времени по вычислительным операторам, оценка вклада MPI-коммуникаций и GPU-редукций, а также сравнительные таблицы ускорений. Эти данные дополняют основной текст отчёта и позволяют более полно оценить масштабируемость и эффективность всех реализованных параллельных схем.

Этап	Время, с	Доля, %
Инициализация	0.028	0.27
Оператор Лапласа	5.476	53.05
Операторы обновления	2.650	25.66
Редукции	2.009	19.46
Завершение	0.170	1.65
Итого	10.324	100.00

Таблица 13: Профиль временных затрат последовательной программы (400×600)

Этап	Время, с	Доля, %
Инициализация	0.008	0.16
Оператор Лапласа	2.809	52.60
Операторы обновления	1.361	25.48
Редукции	1.017	19.03
Завершение	0.141	2.64
Всего	5.341	100.00

Таблица 14: Профиль временных затрат OpenMP-реализации (2 нити на 400×600)

Этап	Время, с	Доля, %
Инициализация	0.005	0.19
Оператор Лапласа	1.394	50.25
Операторы обновления	0.694	25.01
Редукции	0.531	19.12
Завершение	0.143	5.16
Всего	2.774	100.00

Таблица 15: Профиль временных затрат OpenMP-реализации (4 нити на 400×600)

Этап	Время, с	Доля, %
Инициализация	0.057	0.07
Оператор Лапласа	44.273	52.04
Операторы обновления	23.151	27.22
Редукции	17.078	20.08
Завершение	0.528	0.62
Итого	85.067	100.00

Таблица 16: Профиль временных затрат последовательной программы (800×1200)

Этап	Время, с	Доля, %
Инициализация	0.032	0.08
Оператор Лапласа	22.192	52.61
Операторы обновления	11.105	26.33
Редукции	8.316	19.72
Завершение	0.529	1.26
Всего	42.185	100.00

Таблица 17: Профиль временных затрат OpenMP-реализации (2 нити на 800×1200)

Этап	Время, с	Доля, %
Инициализация	0.020	0.09
Оператор Лапласа	11.404	52.74
Операторы обновления	5.501	25.43
Редукции	4.162	19.25
Завершение	0.528	2.44
Всего	21.628	100.00

Таблица 18: Профиль временных затрат OpenMP-реализации (4 нити на 800×1200)

Этап	Время, с	Доля, %
Инициализация	0.234	5.14
Оператор Лапласа	1.727	37.91
Операторы обновления	1.002	22.01
Локальные редукции	1.368	30.03
MPI halo exchange	0.009	0.21
MPI Allreduce	0.047	1.04
Завершение	0.154	3.38
Всего	4.556	100.00

Таблица 19: Профиль временных затрат MPI-реализации (400×600 , $P = 2$)

Этап	Время, с	Доля, %
Инициализация	0.130	5.32
Оператор Лапласа	0.880	36.03
Операторы обновления	0.500	20.49
Локальные редукции	0.691	28.32
MPI halo exchange	0.008	0.32
MPI Allreduce	0.058	2.39
Завершение	0.160	6.53
Всего	2.441	100.00

Таблица 20: Профиль временных затрат MPI-реализации (400×600 , $P = 4$)

Этап	Время, с	Доля, %
Инициализация	0.137	0.38
Оператор Лапласа	14.309	39.98
Операторы обновления	9.164	25.61
Локальные редукции	11.451	32.01
MPI halo exchange	0.030	0.08
MPI Allreduce	0.076	0.21
Завершение	0.563	1.57
Всего	35.783	100.00

Таблица 21: Профиль временных затрат MPI-реализации (800×1200 , $P = 2$)

Этап	Время, с	Доля, %
Инициализация	0.149	0.83
Оператор Лапласа	7.145	39.59
Операторы обновления	4.276	23.69
Локальные редукции	5.630	31.19
MPI halo exchange	0.031	0.17
MPI Allreduce	0.221	1.23
Завершение	0.564	3.13
Всего	18.049	100.00

Таблица 22: Профиль временных затрат MPI-реализации (800×1200 , $P = 4$)

Этап	Время, с	Доля, %
Инициализация	0.231	18.84
Операторы Лапласа	0.081	6.60
- CPU	0.007	0.58
- CUDA	0.074	6.02
Операторы обновления	0.077	6.27
- CPU	0.010	0.81
- CUDA	0.067	5.46
Локальные редукции	0.377	30.73
- CPU	0.021	1.70
- CUDA	0.356	29.03
Копирование Host→Device	0.005	0.40
Копирование Device→Host	0.090	7.34
MPI halo exchange	0.004	0.33
MPI Allreduce	0.014	1.10
Завершение	0.239	19.44
Итого	1.228	100.00

Таблица 23: Профиль временных затрат MPI+CUDA-реализации (400×600 , 1 GPU)

Этап	Время, с	Доля, %
Инициализация	0.227	18.53
Операторы Лапласа	0.054	4.42
- CPU	0.003	0.28
- CUDA	0.051	4.15
Операторы обновления	0.075	6.13
- CPU	0.005	0.41
- CUDA	0.070	5.72
Локальные редукции	0.327	26.73
- CPU	0.022	1.80
- CUDA	0.305	24.93
Копирование Host→Device	0.032	2.59
Копирование Device→Host	0.133	10.87
MPI halo exchange	0.008	0.68
MPI Allreduce	0.031	2.55
Завершение	0.235	19.21
Итого	1.223	100.00

Таблица 24: Профиль временных затрат MPI+CUDA-реализации (400×600 , 2 GPU)

Этап	Время, с	Доля, %
Инициализация	0.218	17.53
Операторы Лапласа	0.040	3.23
- CPU	0.001	0.07
- CUDA	0.039	3.15
Операторы обновления	0.058	4.70
- CPU	0.001	0.09
- CUDA	0.057	4.61
Локальные редукции	0.260	20.96
- CPU	0.019	1.50
- CUDA	0.241	19.46
Копирование Host→Device	0.040	3.26
Копирование Device→Host	0.142	11.44
MPI halo exchange	0.038	3.09
MPI Allreduce	0.118	9.54
Завершение	0.259	20.90
Итого	1.240	100.00

Таблица 25: Профиль временных затрат MPI+CUDA-реализации (400×600 , 4 GPU)

Этап	Время, с	Доля, %
Инициализация	0.311	5.52
Операторы Лапласа	0.534	9.48
- CPU	0.027	0.49
- CUDA	0.507	8.99
Операторы обновления	0.200	3.54
- CPU	0.039	0.69
- CUDA	0.161	2.86
Локальные редукции	2.779	49.31
- CPU	0.046	0.81
- CUDA	2.733	48.50
Копирование Host→Device	0.012	0.21
Копирование Device→Host	0.248	4.40
MPI halo exchange	0.009	0.16
MPI Allreduce	0.030	0.53
Завершение	0.970	17.20
Итого	5.638	100.00

Таблица 26: Профиль временных затрат MPI+CUDA-реализации (800×1200 , 1 GPU)

Этап	Время, с	Доля, %
Инициализация	0.272	6.88
Операторы Лапласа	0.303	7.67
- CPU	0.013	0.34
- CUDA	0.290	7.33
Операторы обновления	0.181	4.58
- CPU	0.020	0.50
- CUDA	0.162	4.09
Локальные редукции	1.349	34.15
- CPU	0.047	1.18
- CUDA	1.302	32.97
Копирование Host→Device	0.095	2.40
Копирование Device→Host	0.362	9.17
MPI halo exchange	0.021	0.54
MPI Allreduce	0.068	1.71
Завершение	0.941	23.89
Итого	3.951	100.00

Таблица 27: Профиль временных затрат MPI+CUDA-реализации (800×1200 , 2 GPU)

Этап	Время, с	Доля, %
Инициализация	0.262	7.73
Операторы Лапласа	0.175	5.18
- CPU	0.007	0.22
- CUDA	0.168	4.96
Операторы обновления	0.169	4.99
- CPU	0.010	0.29
- CUDA	0.159	4.70
Локальные редукции	0.860	25.42
- CPU	0.045	1.33
- CUDA	0.815	24.09
Копирование Host→Device	0.118	3.49
Копирование Device→Host	0.391	11.56
MPI halo exchange	0.034	0.99
MPI Allreduce	0.169	4.99
Завершение	0.948	28.02
Итого	3.384	100.00

Таблица 28: Профиль временных затрат MPI+CUDA-реализации (800×1200 , 4 GPU)